Lecture 01

Introduction to **Object-Oriented Programming**

Object-Oriented Programming in JAVA

Prepared by **Hok Tin**

Content

- Intro. to Java Programming
- Intro. to Object-Oriented Programming
- Java Basics

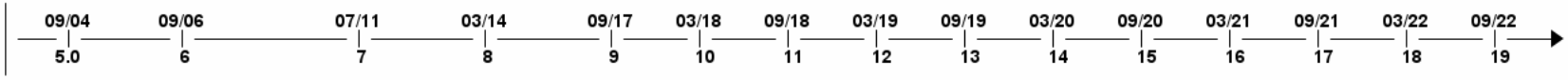
What Is Java?

- Java is an object-oriented programming language developed by Sun Microsystems
- Modeled after C++, the Java language was designed to be small, simple, and portable
- Across platforms and operating systems, both at the source and at the binary level

JDK 19 and 20 build platforms supported by Oracle

OS and CPU	OS Type/Vendor	OS Version	Compiler/Toolchain Type/Vendor	Compiler/Toolchain Version
Linux x86_x64	Oracle Enterprise Linux	6.4/7.x	gcc	11.2.0
Linux aarch64	Oracle Enterprise Linux	8.x	gcc	11.2.0
macOS x86_64	macOS	10.15.x	XCode / clang	12.4
macOS aarch64	macOS	11.2	XCode / clang	12.4
Windows x86_64	Windows	Server 2016	Microsoft Visual Studio	2022 17.1.0

- Java versions released; Java 20 is scheduled for March 2023

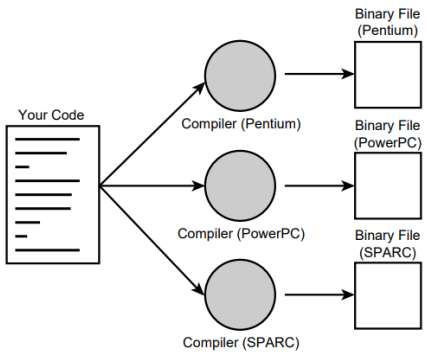


Why Learn Java?

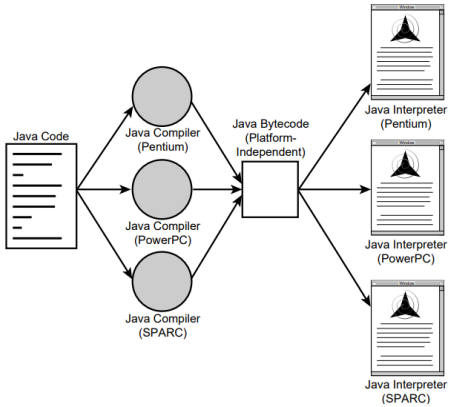
✓ **Java Is Platform-Independent**

Platform independence is one of the most significant advantages that Java has over other programming languages, particularly for systems that need to work on many different platforms. Java is platform-independent at both the ***source*** and the ***binary*** level.

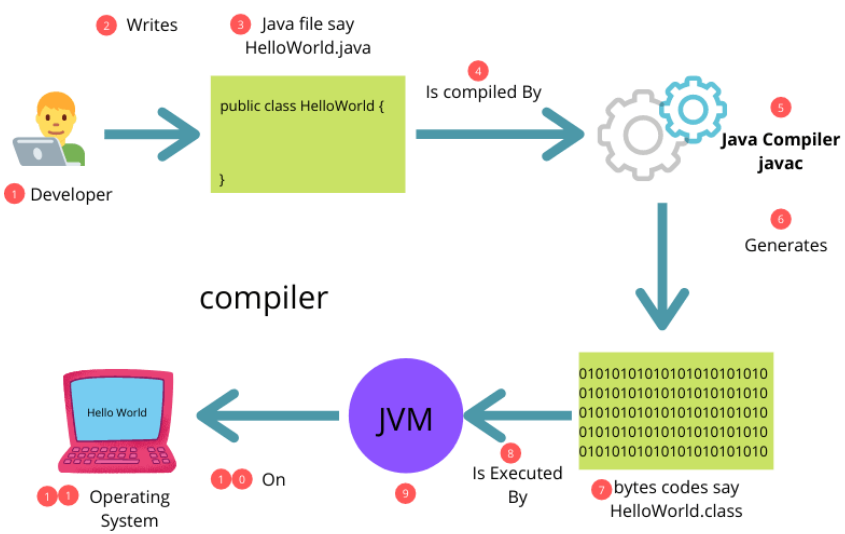
Traditional compiled programs



Java programs



How Java Works



Why Learn Java?

✓ **Java Is Object-Oriented**

- OOP technique is merely a way of ***organizing programs***, and it can be ***accomplished using any languages***
- Enables you to take full advantage of ***object-oriented methodology*** and its capabilities of ***creating flexible***, ***modular programs*** and ***reusing code***.
- ***Inherited from C++,*** the language on which it is based, but it borrows many concepts from other object-oriented languages as well
- Java includes a set of class libraries that provide ***basic data types***, system ***input and output capabilities***, and other utility functions
- These basic classes are part of the Java Development Kit (**JDK**), which also has classes to support ***networking***, common ***Internet protocols***, and ***user interface toolkit functions***

Why Learn Java?

✓ **Java Is Easy to Learn**

- One of Java's initial design goals was to be ***small*** and ***simple***, and therefore ***easier to write***, ***easier to compile***, ***easier to debug***, and, best of all, ***easy to learn***
- Java still has a great deal of power and ***flexibility***
- Java is modeled after C and C++, and ***much of the syntax*** and ***object-oriented structure***
- Most of the ***more complex parts of C and C++ have been excluded*** from Java, making the language simpler without sacrificing much of its power

Create a Java Application

Let's start by creating a simple "***Hello World***" Java application:

```
1  public class HelloWorld{
2      public static void main(String []args){
3          System.out.println("Hello, World!");
4      }
5  }
```

Start writing your code from here

This program has **two main parts**:

- All the program is enclosed in a **Class** definition—here, a class called ***HelloWorld***
- The body of the program (here, just the one line) is contained in a routine called **main()**. In Java applications, as in a C or C++ program, **main()** is the first routine that is run when the program is executed.

→ Now, let's compile the source file using the Java compiler. In Sun's **JDK**, the Java compiler is called **javac**.

Object-Oriented Programming

What is Object-Oriented Programming (OOP)?

OOP is one of the bigger programming buzzwords of recent years, and you can spend years learning all about object-oriented programming methodologies and how they can make your life easier than the Old Way of programming

You'll get an overview of object-oriented programming concepts in Java:

- What ***classes*** and ***objects*** are, and how they relate to each other
- The two main parts of a class or object: its ***behaviors*** and its ***attributes***
- Class ***inheritance*** and how inheritance affects the way you design your programs
- Some information about ***packages*** and ***interfaces***

Object-Oriented Programming

The reasons we need OOP

Primitive Data Types

Data Type	Type Definition
Integers	byte, short, int, long
Floating-Point	float, double
Character	char
Boolean	boolean

Datatype Variable

Primitive datatype:

```
double  money;
char  gender;
```

Customized datatype:

```
Person  personObj;
Student  stuObj;
Employee  empObj;
BankAccount  accObj;
```

Class Object

Class

- **Person** — To describe characteristics of a person (name, sex, age)
- **Student** — To describe characteristics of a student (sid, name, sex, age, score, etc…)
- **BankAccount** — To describe properties of bank account of a customer (account_id, name, balance, account_type, etc…)

☞ You can design whatever datatype you want…

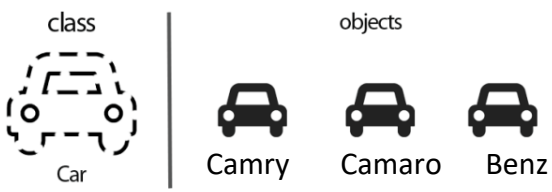
- Your own customized datatype is called **Class**
- The variable created from your Class is called **Object**

Object-Oriented Programming

Class and Object

- A **class** is a template for multiple objects with similar features. Classes embody all the features of a particular set of objects
- When you write a program in an object-oriented language, you don't define actual objects. You define classes of objects.

For example: you want to create your ***own datatype*** (**Class**) to represent cars:



Create an empty class named "**Car**"

```
class Car {

}
```

Create **objects** by "Car" class

```
Car camry;
Car camaro;
Car benz;
```

Object-Oriented Programming

Attributes and Behavior

Every class you write in Java is generally made up of two components: **attributes** and **behavior**

❑ Attributes

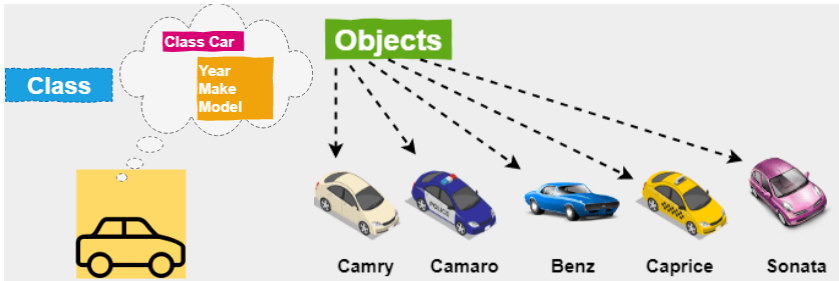
Attributes are the individual things that differentiate one object from another and determine the ***appearance***, ***state***, or other ***qualities*** of that object.

Let's create a theoretical class called ***Motorcycle***. The attributes of a motorcycle might include the following:

- ❑ **Color**: red, green, silver, brown
- ❑ **Style**: cruiser, sport bike, standard
- ❑ **Model**: Honda, BMW, Bultaco

- **Attributes** are defined by ***variables***; in fact, you can consider them analogous to global variables for the entire object.
- Each instance of a class can have different values for its variables, each variable is called an instance variable

☞ **For example**: you want to create your own datatype (Class) ***with attributes/properties*** to represent cars:



Create a class with <u>attributes</u>

```
1  class Car {
2      int year;
3      String make;
4      String model;
5  }
```

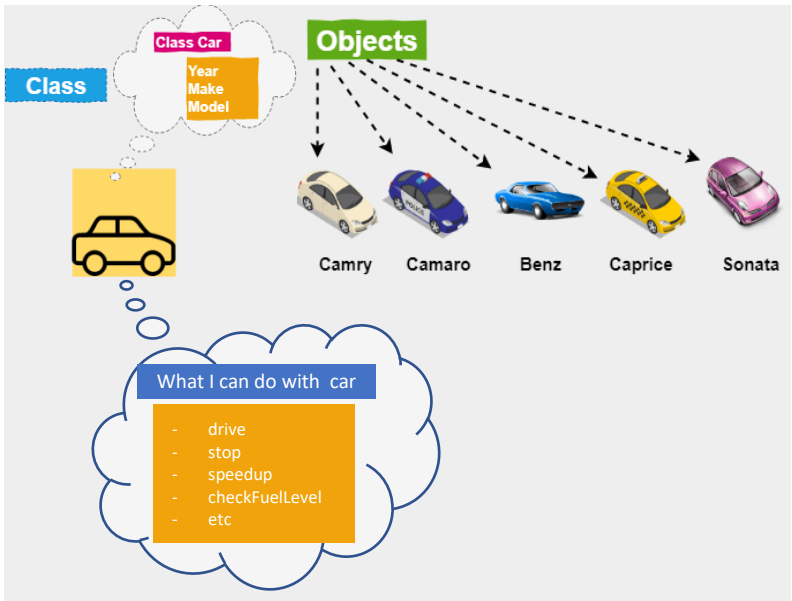
Attributes
(variables of a class)

Object-Oriented Programming

❑ Behavior

- A class's behavior determines what instances of that class do when their internal state changes or when that instance ***is asked to do something*** by another class or object
- ***Behavior*** is the way objects can do anything to themselves or have anything done to them
- To define an object's behavior, you create methods, which look and behave just ***like functions in other languages*** but are defined inside a class.
- Java ***does not have*** functions defined ***outside classes*** (as C++ does)

☞ **For example**: you want to create a Class with attributes ***and Behaviors*** to represent cars:



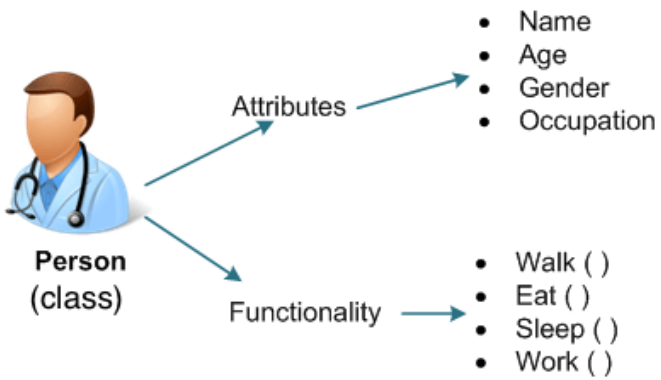

```
class Car {
    int speed;
    String model;
    bool isRunning;
    int fuelLevel;

    void drive(){
        speed = 1;
    }
    void stop(){
        speed = 0;
    }
    void speedUp(){
        speed = speed + 1;
    }
    int checkFuel(){
        return fuelLevel;
    }
}
```

Attributes
(variables of a class)

Behaviors
(Function or method in a class)

Object-Oriented Programming

✓ **Another example of a class consisting of Attributes and Behaviors**



Create many objects from a **Person** class

```
Person doctor;
Person teacher;
Person student;
```

Object#1: doctor

Name: Tola
Age: 27
Gender: M
Occupation: "dentist"

Walk()
Eat()
Work()
Sleep()

Object#2: teacher

Name: Makara
Age: 26
Gender: F
Occupation: "math teacher"

Walk()
Eat()
Work()
Sleep()

Object#3: student

Name: Vicheka
Age: 18
Gender: M
Occupation: "student"

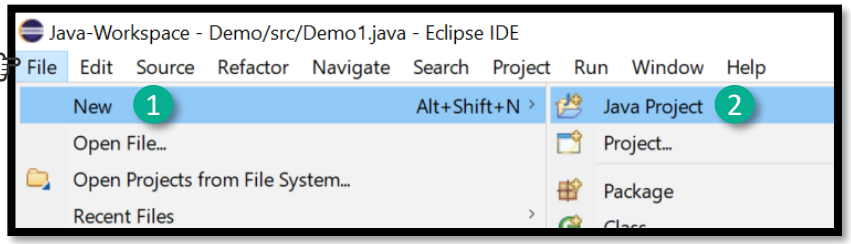
Walk()
Eat()
Work()
Sleep()

Java Basics

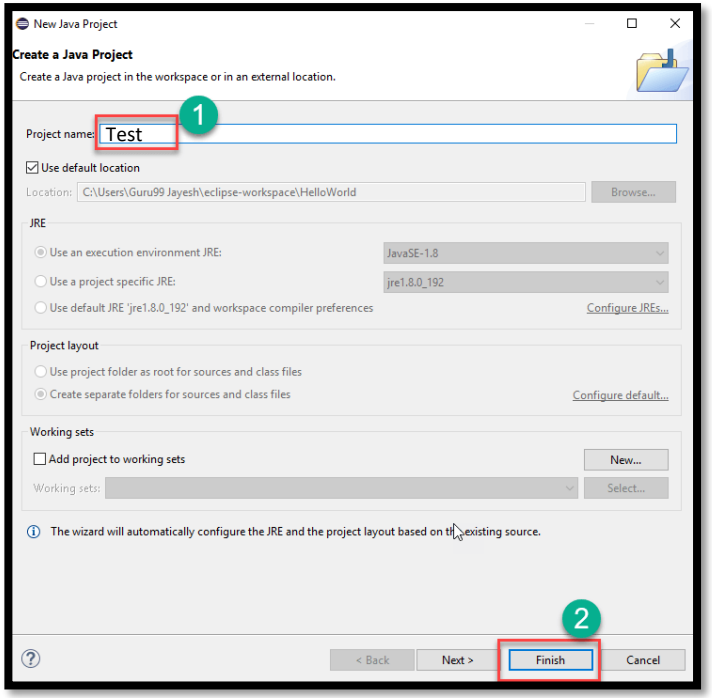
❑ Get started with your first project:

1. Download/Install Java JDK (Java Development Kit) on your machine
2. Download/Install Eclipse IDE to Run Java
3. Create a java project in eclipse: **`File->New->Java Project`**

Creating:



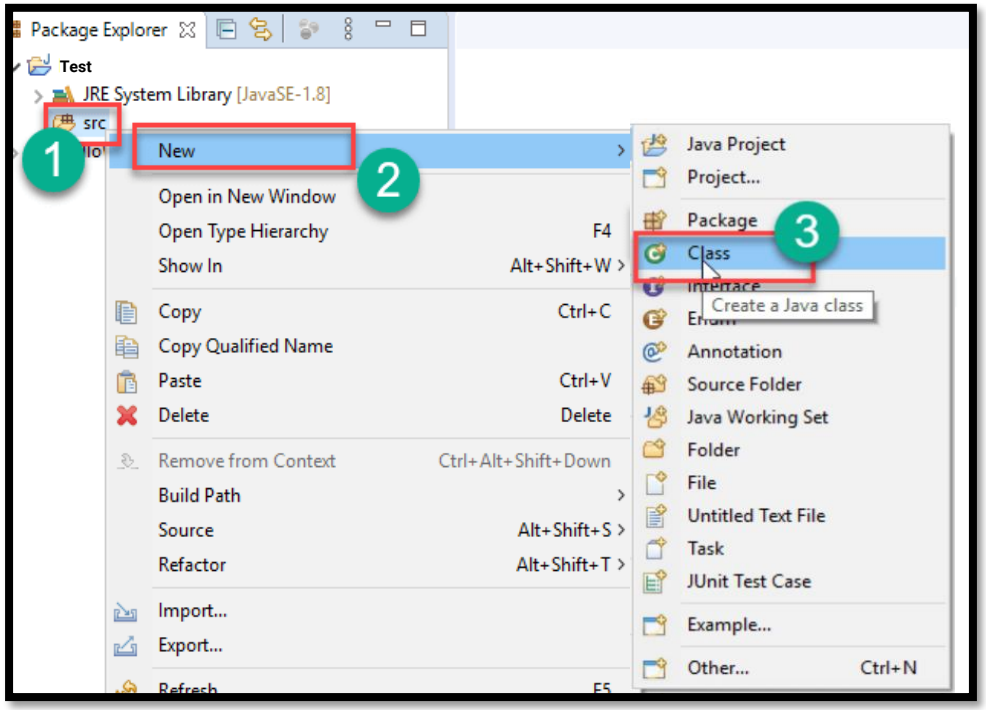
Naming:

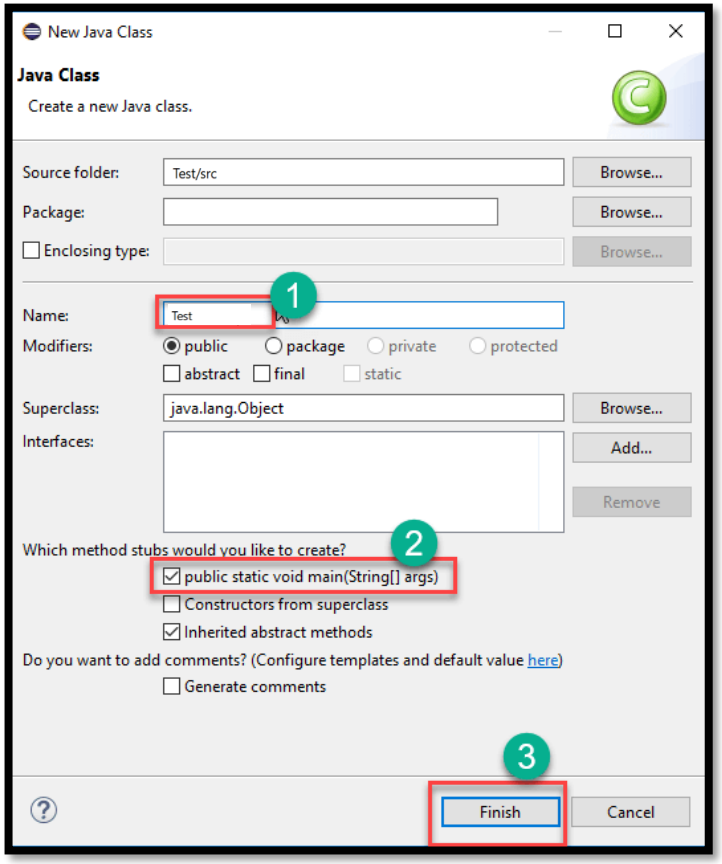


Java Basics

❑ Get started with your first project:

1. Download/Install Java JDK (Java Development Kit) on your machine
2. Download/Install Eclipse IDE to Run Java
3. Create a java project in eclipse: **`File->New->Java Project`**
4. Create a java class in "src" folder

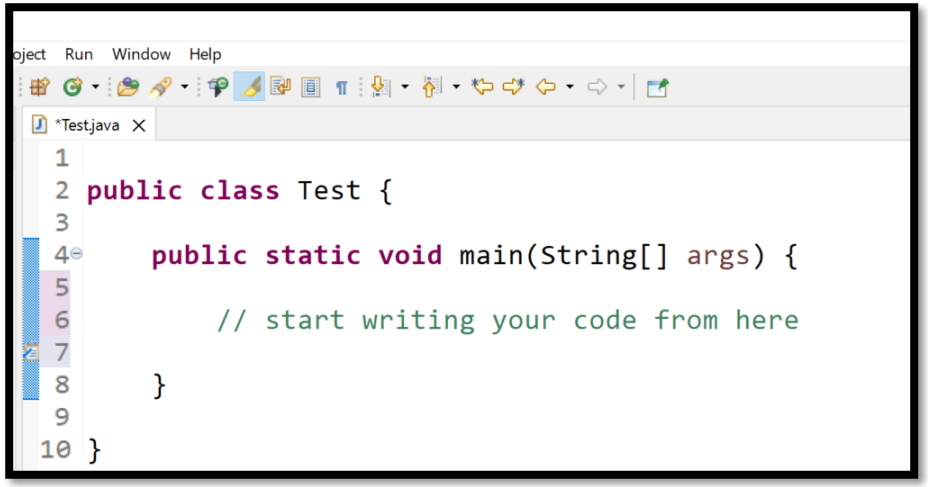




Java Basics

❑ Get started with your first project:

1. Download/Install **Java JDK** (Java Development Kit) on your machine
2. Download/Install **Eclipse IDE** to Run Java
3. Create a java project in eclipse: **File->New->Java Project**
4. Create a java class in "**src**" folder
5. Write your code in "**main**" **function** in the created class



Java Basics

❑ Statements

- A ***statement*** is the simplest thing you can do in Java; a statement forms a single Java operation

```java
import java.util.Scanner;
public class Test{
    public static void main(String []args){
        int a = 1;
        int b = 5;
        int c = a + b;
        System.out.println("The result is: " + c);
    }
}
```

The most important thing to remember about Java statements is that each ***one ends with a semicolon***. (Forget the semicolon and your Java program won't compile)

❑ Comments

Comments can be used to explain Java code, and to make it more readable. It can also be used to prevent execution when testing alternative code.

- **Single-line Comments**
- **Multi-line/Block Comments**

```java
public class Test {
    public static void main(String[] args) {
        // This is a comment
        System.out.println("Hello World");

        System.out.println("Hello World"); // This is a comment

        /* The code below will print the words Hello World
        to the screen, and it is amazing */
        System.out.println("Hello World");
    }
}
```

Java Basics

❑ Data types

Type	Example	Accepted values
Whole number, i.e., `int`	`int value = 4;`	An integer can contain whole numbers whose values lie between -2147483648 and 2147483647.
Floating-point number, i.e., `double`	`double value = 4.2;`	Floating-point numbers contain decimal numbers, with the greatest possible value being approximately 2^{1023}
`String`	`String text = "Hi!";`	A string can contain text. Strings are enclosed in quotation marks.
True or false value, i.e., `boolean`	`boolean right = true;`	A boolean contains either the value `true` or `false`.

- There are four Java integer types, each with different ranges of values as listed below

Type	Size	Range
`byte`	8 bits	–128 to 127
`short`	16 bits	–-32,768 to 32,767
`int`	32 bits	–2,147,483,648 to 2,147,483,647
`long`	64 bits	–9223372036854775808 to 9223372036854775807

- There are two floating-point types: **float** (32 bits, single-precision) and **double** (64 bits, double-precision)
- The **char** type is used for individual characters.
- The **boolean** type can have one of two values, **true** or **false**

Java Basics

❑ Variables

- You can string together variable names with the same type:

```
int x, y, z;
String firstname, lastName;
```

- You can also give each variable an initial value when you declare it:

```
int myAge, mySize, numShoes = 28;
String myName = "Tola";
boolean isTired = true;
int a = 4, b = 5; c = 6;
```

✍ Notes on Variable Names

- Variable names in Java can ***start with a letter***, an underscore (_), or a dollar sign (**$**)
- They ***cannot*** start with a number
- After the first character, your variable names can include any letter or number
- Symbols, such as **%**, *****, **@**, and so on, are often ***reserved for operators*** in Java, so be careful when using symbols in variable names
- Java language is ***case-sensitive***, which means that uppercase letters are different from lowercase letters

```
Button theButton;
long reallyBigNumber;
boolean currrentWhetherStateOfPlanetXShortVersion;
```

Java Basics

❑ String in Java

- In Java, String is a class used to be as a datatype of a sequence of characters

```java
public class Test {
    public static void main(String[] args) {
        String str = "Java programming";

        // print string
        System.out.println(str);   // output: Java programming

        // concat string with + expression
        String greet = "Hello! World";
        System.out.println("String: " + greet);

        // get the length of greet
        int length = greet.length();
        System.out.println("Length: " + length);

        // join two strings
        String first = "Java ";
        String second = "Programming";
        String third = "Programming";
        String joinedString = first.concat(second);
        System.out.println("Joined String: " + joinedString);

        // compare first and third strings
        boolean result1 = first.equals(second);
        System.out.println("Strings first and second are equal: " + result1);

        // compare second and third strings
        boolean result2 = second.equals(third);
        System.out.println("Strings second and third are equal: " + result2);
    }
}
```

Output:

```
Java programming
String: Hello! World
Length: 12
Joined String: Java Programming
Strings first and second are equal: false
Strings second and third are equal: true
```

Java Basics

❑ Simple arithmetic

```java
class Test {
    public static void main (String[] args) {
        short x = 6;
        int y = 4;
        float a = 12.5f;
        float b = 7f;

        System.out.println("x is " + x + ", y is " + y);
        System.out.println("x + y = " + (x + y));
        System.out.println("x - y = " + (x - y));
        System.out.println("x / y = " + (x / y));
        System.out.println("x % y = " + (x % y));

        System.out.println("a is " + a + ", b is " + b);
        System.out.println("a / b = " + (a / b));
    }
}
```

Output:

```
x is 6, y is 4
x + y = 10
x - y = 2
x / y = 1
x % y = 2
a is 12.5, b is
7.0
a / b = 1.7857143
```

Java Basics

❑ Printing

- The print command **System.out.println("Hello world");** prints the text "Hello world".

```java
public class Example {
    public static void main(String[] args) {
        // Here goes the statements used by the program
        System.out.println("Hello world!");
    }
}
```

Output:

```
Hello world!
```

- Different type of common print functions (**`print, println, printf`**)

```java
public class Test {
    public static void main(String[] args) {
        int num = 122;
        char ch = 'A';
        String str = "Oracle";  //
        double d = 190.98;
        float f = 3.14f;

        System.out.println(); //prints nothing but throws the cursor to the next line
        System.out.println(num); //prints integer
        System.out.println(ch); //prints character
        System.out.print(str+"\n");
        System.out.print(d +"\n");
        System.out.print(f+"\n");
        System.out.printf("'%s' %n", "javatpoint");  // %s for string, %n for new line
        System.out.printf("'%S' %n", "Jack");
    }
}
```

Output:

```
122
A
Oracle
190.98
3.14
'javatpoint'
'JACK'
```

Java Basics

❑ Options of Print function

- PRINTF FORMAT SPECIFIERS

Specifier	Type
%c	character
%d	integer
%f	floating-point number
%s	a string of characters
%%	print a percent sign
%e	exponential floating-point number

- RIGHT-JUSTIFYING PRINTF

Code	Result
printf("%3d", 0);	0
printf("%5d", 678);	678
printf("%10d", -10);	-10

- LEFT-JUSTIFYING PRINTF

Code	Result
printf("%-3d", 0);	'0 '
printf("%-5d", 678);	'678 '
printf("%-20d", -10);	'-10 '

Java Basics

❑ Options of Print function

- THE PRINTF INTEGER ZERO-FILL OPTION

Code	Result
printf("%03d", 1);	001
printf("%05d", 678);	00678
printf("%04d", -10);	-010

- PRINTF SPECIAL CHARACTERS

\n	newline, or linefeed
\t	tab
\\	backslash

- FORMATTING FLOATING POINT NUMBERS WITH PRINTF

Description	Code	Result
One position after the decimal	printf("'%.1f'", 99.5678);	'99.6'
Two positions after the decimal	printf("'%.2f'", 99.5678);	'99.57'
Ten-wide, two positions after the decimal	printf("'%10.2f'", 99.5678);	' 99.57'
Ten-wide, two positions after the decimal, zero-filled	printf("'%010.2f'", 99.5678);	'0000010.35'
Ten-wide, two positions after the decimal, left-justified	printf("'%–10.2f'", 99.5678);	'99.57 '

Good luck 👌

Refs:

- ❑ Teach Yourself JAVA in 21 Days, Laura Lemay and Charles L. Perkins
- ❑ https://www.guru99.com/install-eclipse-java.html
- ❑ https://phoenixnap.com/kb/install-java-windows
- ❑ https://learnjavacoding.com/definitions/print-options-java/
- ❑ https://www.tutorialspoint.com/online_java_compiler.php
- ❑ https://www.javatpoint.com/how-to-print-in-java
- ❑ https://java-programming.mooc.fi/part-1